

Object-oriented design

MohammadAli Shojaie

)Based on David Grellscheid's slides)

Programming paradigm examples

Structured /

Non-Structured Declarative

/ Imperative Procedural

Object-oriented

Functional

(Almost) any style can be implemented in any language

Grady Booch

“Object-oriented analysis and design”

2nd edition

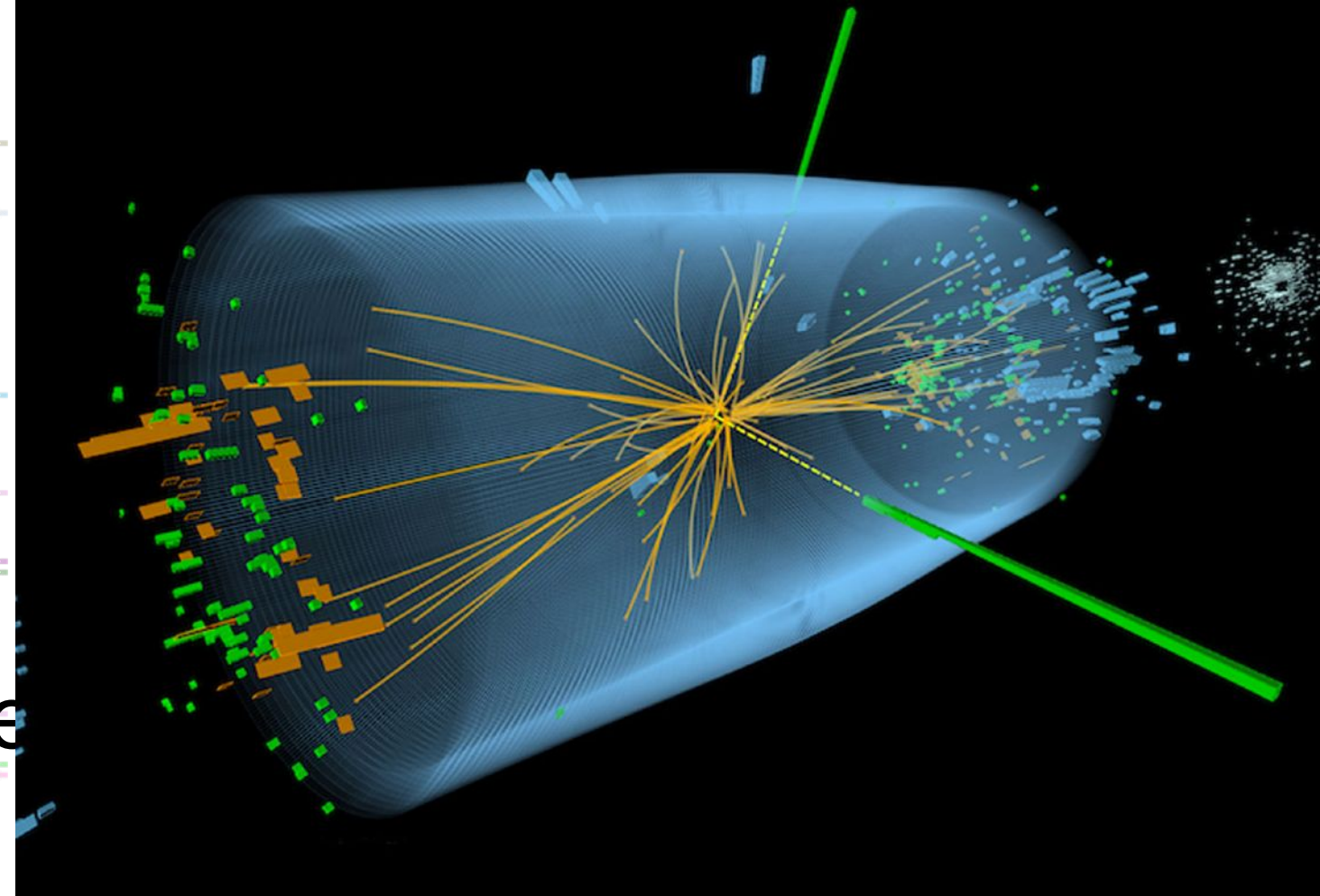
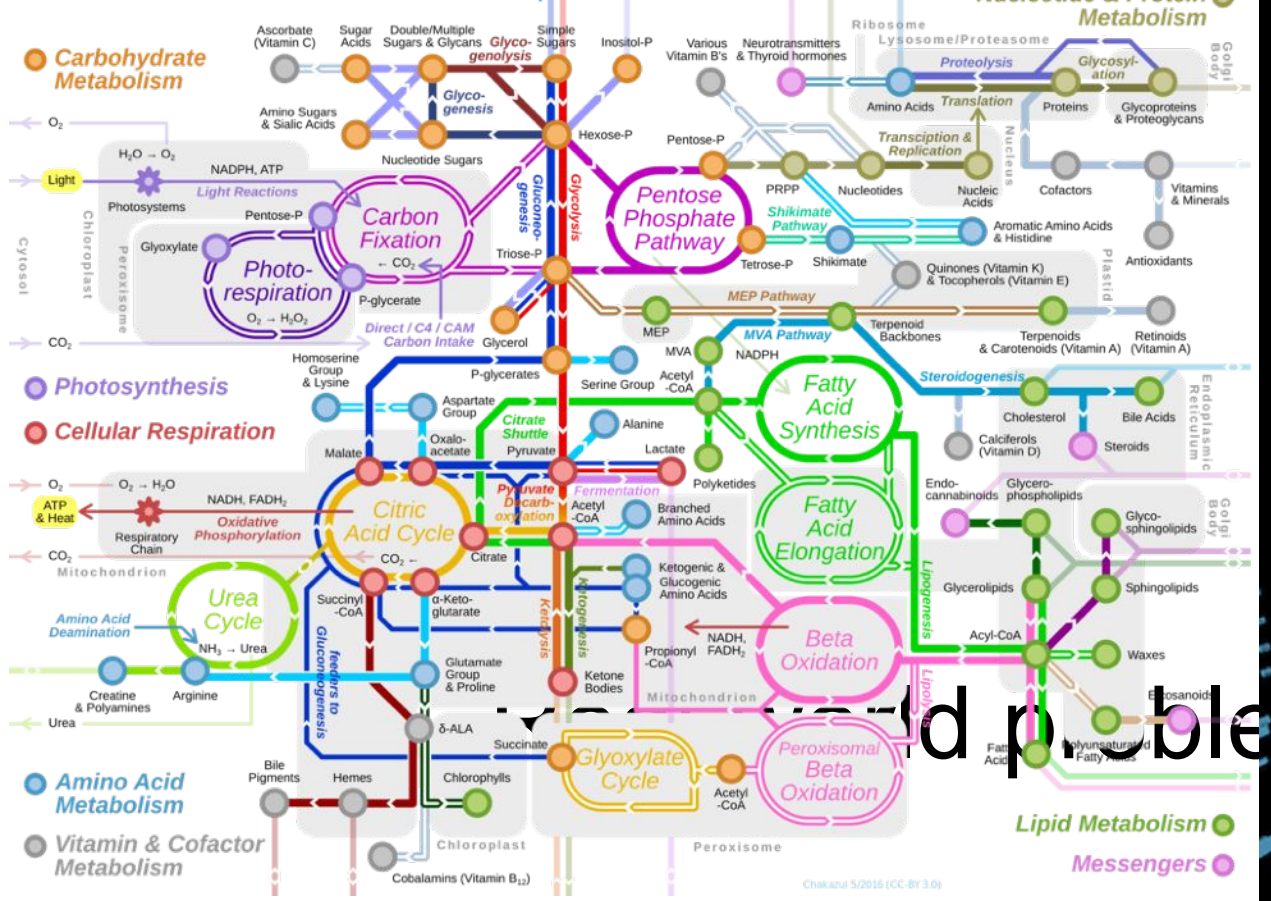
Addison-Wesley, 1994

Main goal: manage
complexity

Different approaches, OO is just one of them!

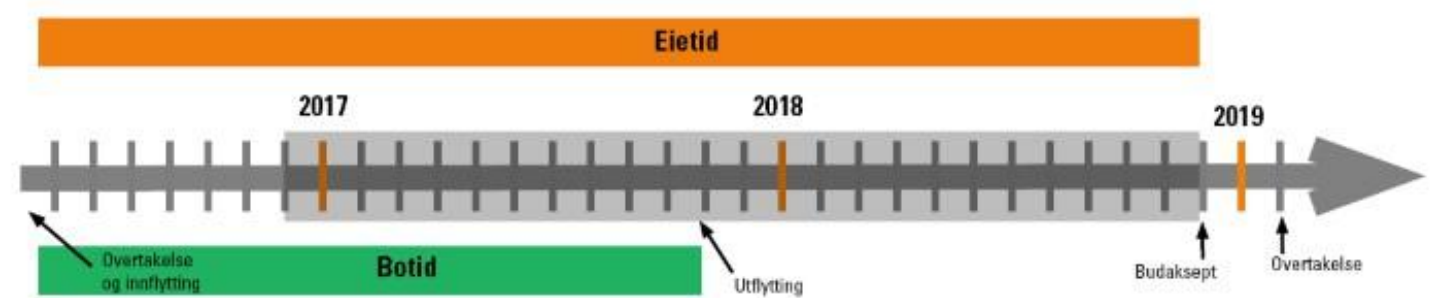
see e.g. Haskell for a completely different approach to
complexity handling: functional programming

Metabolic Map



Må du skatte av boligsalget?

Om du må skatte henger sammen med bo og eietid på boligen du selger. Slik finner du ut dette:

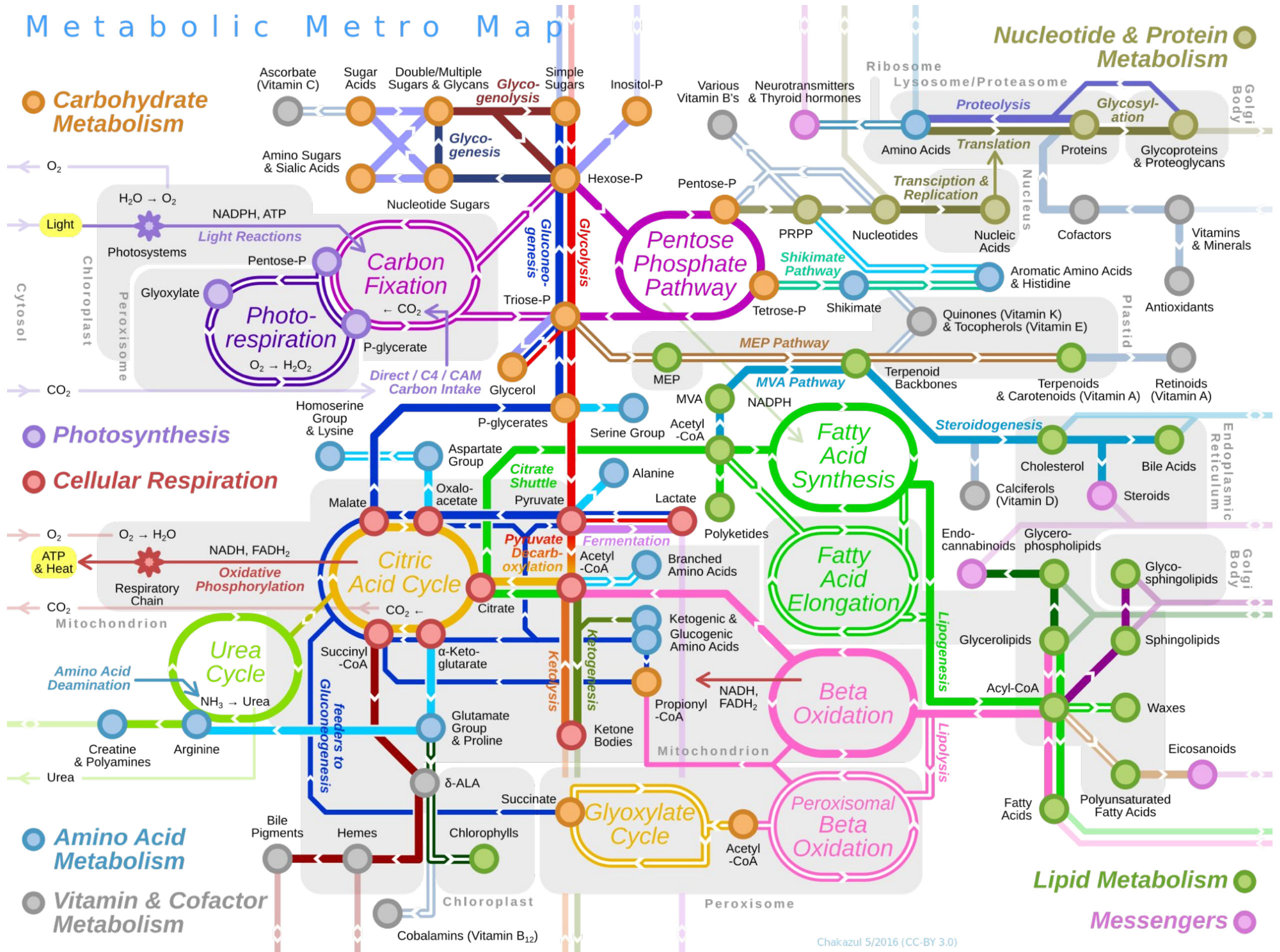


1. Finn datoen for når du aksepterte budet for salg på boligen
2. Tell 24 måneder tilbake fra denne datoen (grått felt i figuren)
3. Har du bodd i boligen i mer enn 12 av disse 24 månedene?
 - Da er salget ikke skattepliktig!
4. Har du ikke bodd i boligen i mer enn 12 av disse 24 månedene?
 - Da er salget mest sannsynlig skattepliktig, og du må skatte av gevinsten.



- * Complexity of the problem domain
external; requires software maintenance, evolution, preservation
- * Development process
impossible for one developer to understand large projects completely
- * Software is boundlessly flexible
able to work at any level of abstraction; no fixed quality standards
- * Behaviour of discrete system
natural world physics is local and continuous
program state is not: combinatoric, small change -> large effect

Metabolic Metro Map



- * Complexity is hierarchical grouping of subsystems, down to elementary components
- * Choice of elementary blocks is mostly arbitrary
- * Links and interactions within a component are much stronger than between components
- * Hierarchy uses only a few different subsystems in different combinations

- * Deal with complexity by
- * decomposition Algorithmic decomposition:
- * which steps in which order?

OO decomposition:

which “real-world” entities are involved? how do they relate to each other?

OO main ideas

Encapsulation and information hiding

Inheritance

Polymorphism

Encapsulation

Program state not globally

modifiable State is kept in

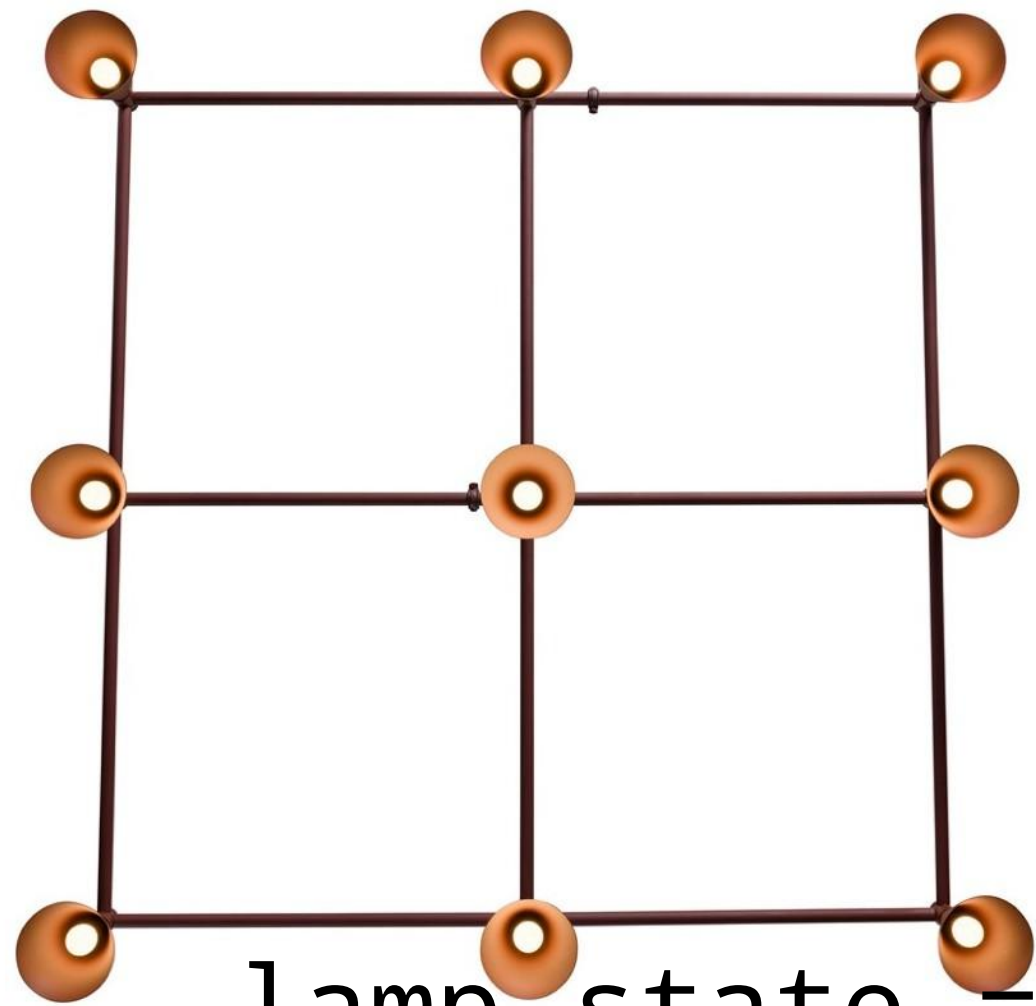
“real-world” objects Object methods

control state change **Object state**

always stays consistent



Topic 1: object state



```
lamp_state = [0,1,1,0,0,0,1,1,1]
```

```
lamp_state = [[0,1,1],[0,0,0],[1,1,1]]
```

```
lamp_state[1][2] == 1
```

Also need two angles for the pointing direction:

```
thetas = [[0.3, 0.4, 0.5],[...],[...]]
```

```
phis = [[0.7, 1.1, 0.0],[...],[...]]
```

Parallel lists are clumsy to use

```
lamp_state = [ [ 0, 1, 1], [...], [...]  
thetas = [ [0.3, 0.4, 0.5], [...], [...]  
phis = [ [0.7, 1.1, 0.0], [...], [...]
```

One possible solution: group the other way

```
lamps = [ [  ,  ,  ], [  ,  ,  ], [  ,  ,  ]]
```

Now,  represents one lamp

Parallel lists are clumsy to use

3 arrays of

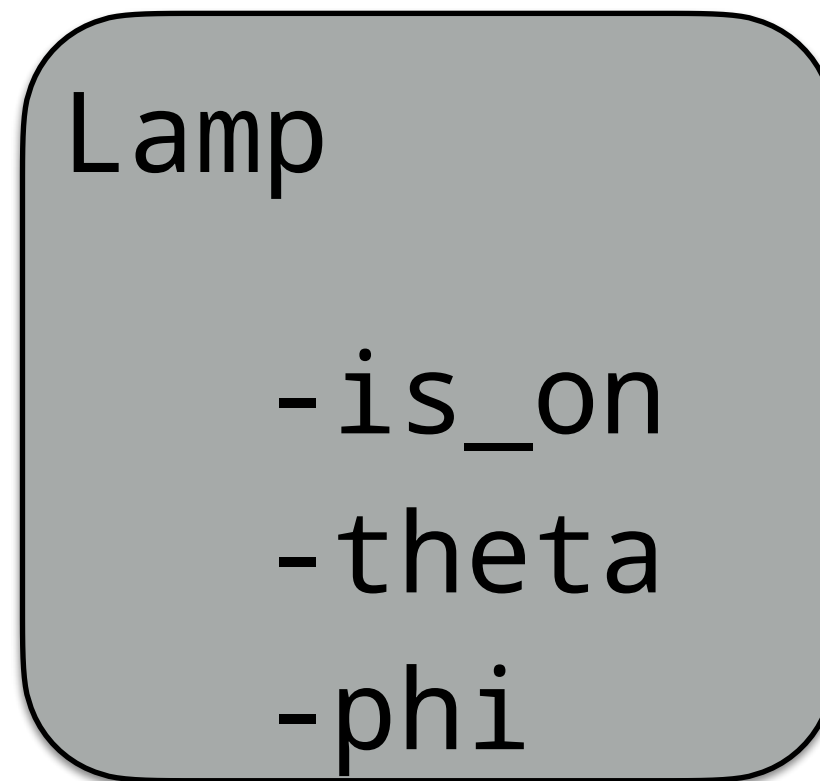
```
lamp_state = [0, 1, values [...], [...]]  
thetas = [0.3, 0.4, 0.5, [...], [...]]  
phis = [0.7, 1.1, 0.0, [...], [...]]
```

One possible solution: group the other way

```
lamps = [[], [], [
```

an array of lamps

Now,  represents one lamp



is_on = 1
theta = 0.1
phi = 0.7

is_on = 0
theta = 0.3
phi = 0.1

is_on = 1
theta = 0.2
phi = 0.37

is_on = 1
theta = 0.9
phi = 1.9

...

Class

Lamp

-is_on
-theta
-phi

attributes /
membervariables

Constructor

is_on = 1
theta = 0.1
phi = 0.7

object

is_on = 0
theta = 0.3
phi = 0.1

object

is_on = 1
theta = 0.2
phi = 0.37

object

is_on = 1
theta = 0.9
phi = 1.9

object

...



```
class Lamp:
```

Constructor

```
    def init (self, on=0, th=0, ph=0):
```

attributes

```
        self.is_on = on
```

```
        self.theta = th
```

```
        self.phi = ph
```

Objects

```
l1 = Lamp(1, 0.4, 0.7)
```

```
l2 = Lamp(0, 1.1, 0.3)
```

```
l3 = Lamp()
```

```
print(l1.is_on)
```

```
print(l2.is_on)
```

```
print(l2.theta)
```




```
public class Lamp {  
    int isOn;  
    double theta;  
    double phi;  
  
    public Lamp(int on, double th, double ph) {  
        isOn = on;  
        theta = th;  
        phi = ph;  
    }  
}
```

attributes

Constructor

objects

```
Lamp l1 = new Lamp(1, 0.4, 0.7);
```

```
Lamp l2 = new Lamp(0, 1.1, 0.3);
```

```
System.out.println(l1.isOn);
```

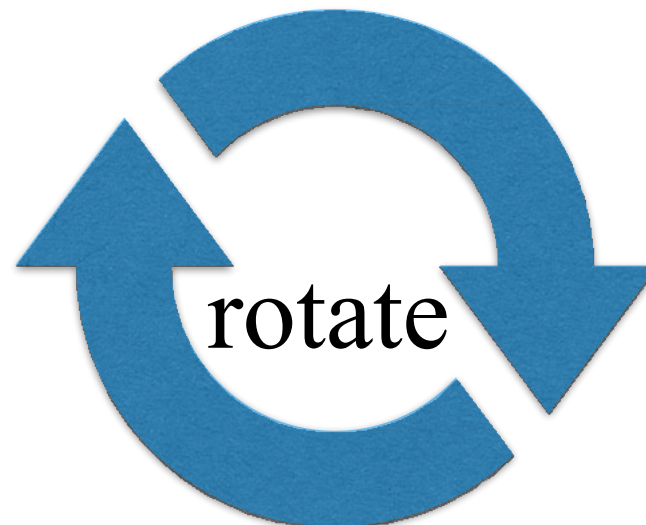
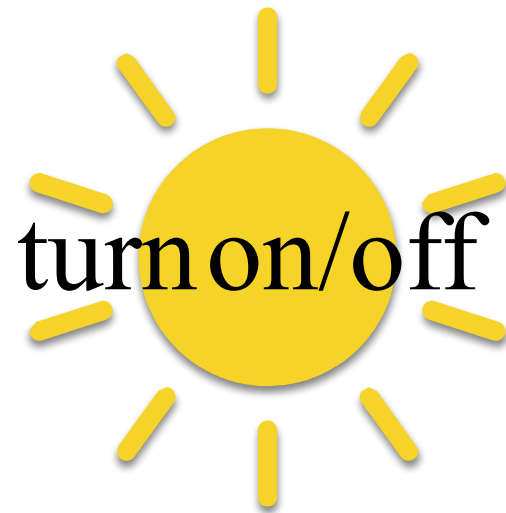
```
System.out.println(l2.theta);
```



Topic 2: object behaviour

We created objects with internal state.

What about lamp behaviours?





```
class Lamp:
```

Constructor

```
def init (self, on=0, th=0, ph=0):  
    self.is_on = on
```

attributes

```
    self.theta = th  
    self.phi = ph
```

```
def turn_on(self):  
    self.is_on = 1
```

```
def turn_off(self):  
    self.is_on = 0
```

```
def rotate(self, angle):  
    self.phi += angle
```

methods /
member functions



```
public class Lamp {  
    int isOn;  
    double theta;  
    double phi;
```

attributes

```
    public Lamp(int on, double th, double ph) {  
        isOn = on;  
        theta = th;  
        phi = ph;  
    }
```

Constructor

```
    public void turnOn() { isOn = 1; }  
    public void turnOff() { isOn = 0; }  
    public void rotate(double angle) { phi += angle; }  
    public void tilt(double angle) { theta += angle; }  
}
```

methods

Object methods allow us to use language from the problem domain rather than basic types:

```
lamp_A = Lamp(1, 0.4, 0.7)  
  
lamp_A.turn_off()  
lamp_A.rotate(0.2)
```

```
Lamp lampA = new Lamp(1, 0.4, 0.7);  
  
lampA.turnOff();  
lampA.rotate(0.2);
```

Object

- * **State:** inner structure with current values
- * **Behaviour:** external interaction and state changes (construct / destruct // modify / select / iterate)
- * **Identity:** distinct to all other objects

It's not the name, one object can have many names!

Identity considerations are relevant when looking at copying, lifetime and ownership behaviour.

Class

Objects with common structure and behaviour belong to a **class**. The class defines both.

An object is an **instance** of a class.

Abstraction

- * Outside view of the object
- * Focus on relevant details, ignore others
- * Defined distinction to other objects

No surprises, no unexpected side behaviour

Abstraction

- * Identify object invariants, properties that must be true at any time
- * Operations have pre- and post-conditions, they must be satisfied
- * Objects should never enter inconsistent state

Encapsulation

- * separates object's tasks from each other
- * actual implementation of the abstraction is hidden
- * allows isolated implementation changes
- * internal design changes in the objects do not impact the users of the objects

Encapsulation

- * Abstractions only work well if implementation is encapsulated!

Modularity

- * Grouping of classes into functionally related units. Modules should be loosely coupled externally.
- * “Physical” collection of units in files, rather than abstract connections
- * Difficult to get right first time, may need several redesigns during development

Hierarchy

- * Abstractions form hierarchies
- * Helps to think about the useful levels

Two main kinds:

- * “is-a”: cat is an animal; oak is a plant
- * “has-a”: car has an engine; house has a door

Hierarchy: “is-a”

- * Modelled by inheritance
- * Common functionality moves to the top;
applies to all classes down the
hierarchy

Easy re-use of code alone is **not**
a good reason for inheritance

Hierarchy: “has-a”

- * Modelledby aggregation
- * Objectshave other objects as member variables

... main message

...

Hierarchy

- * Abstractions form hierarchies
- * Helps to think about the useful levels

Two main kinds:

- * “is-a”: cat is an animal; oak is a plant
- * “has-a”: car has an engine; house has a door

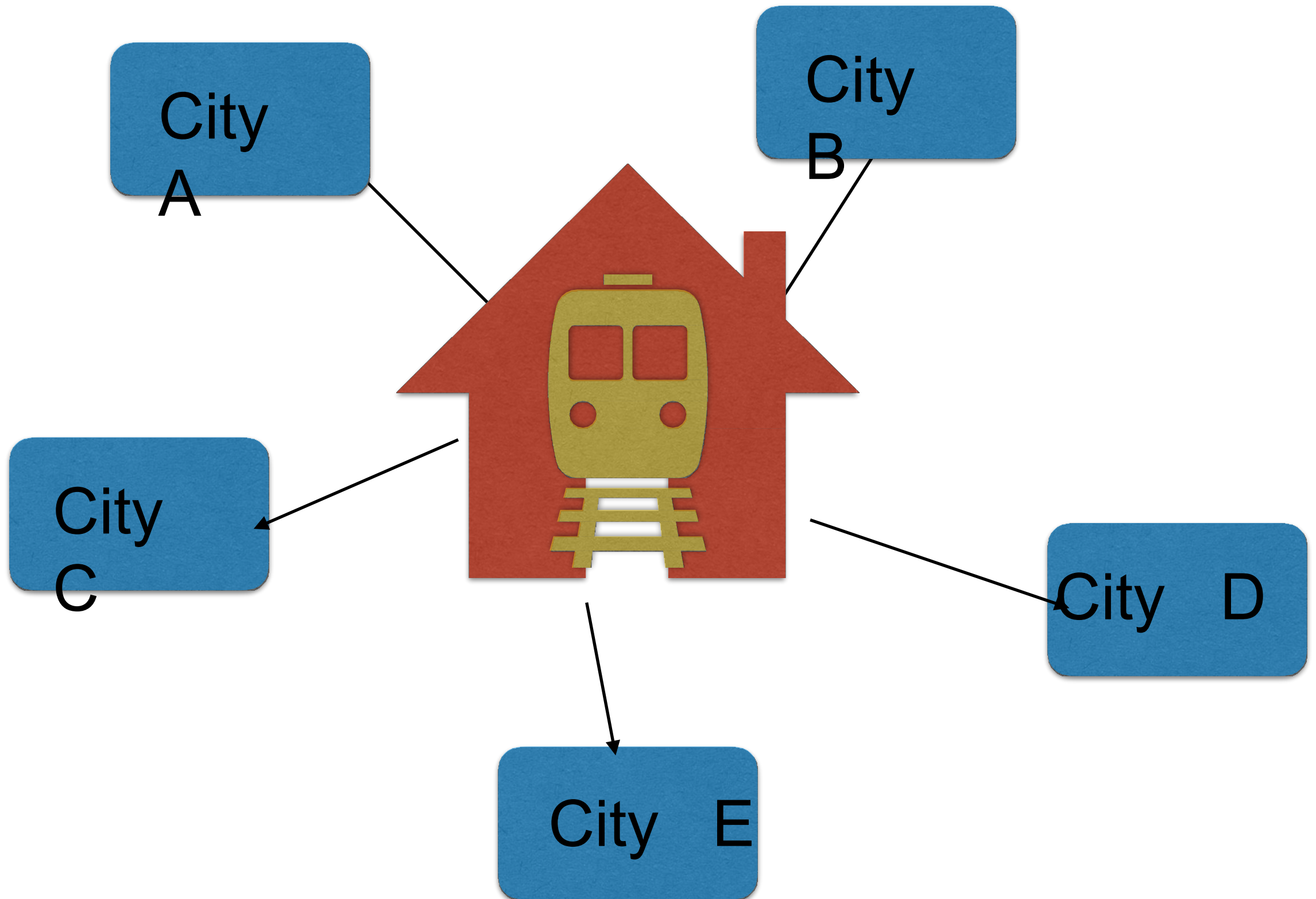
An object has

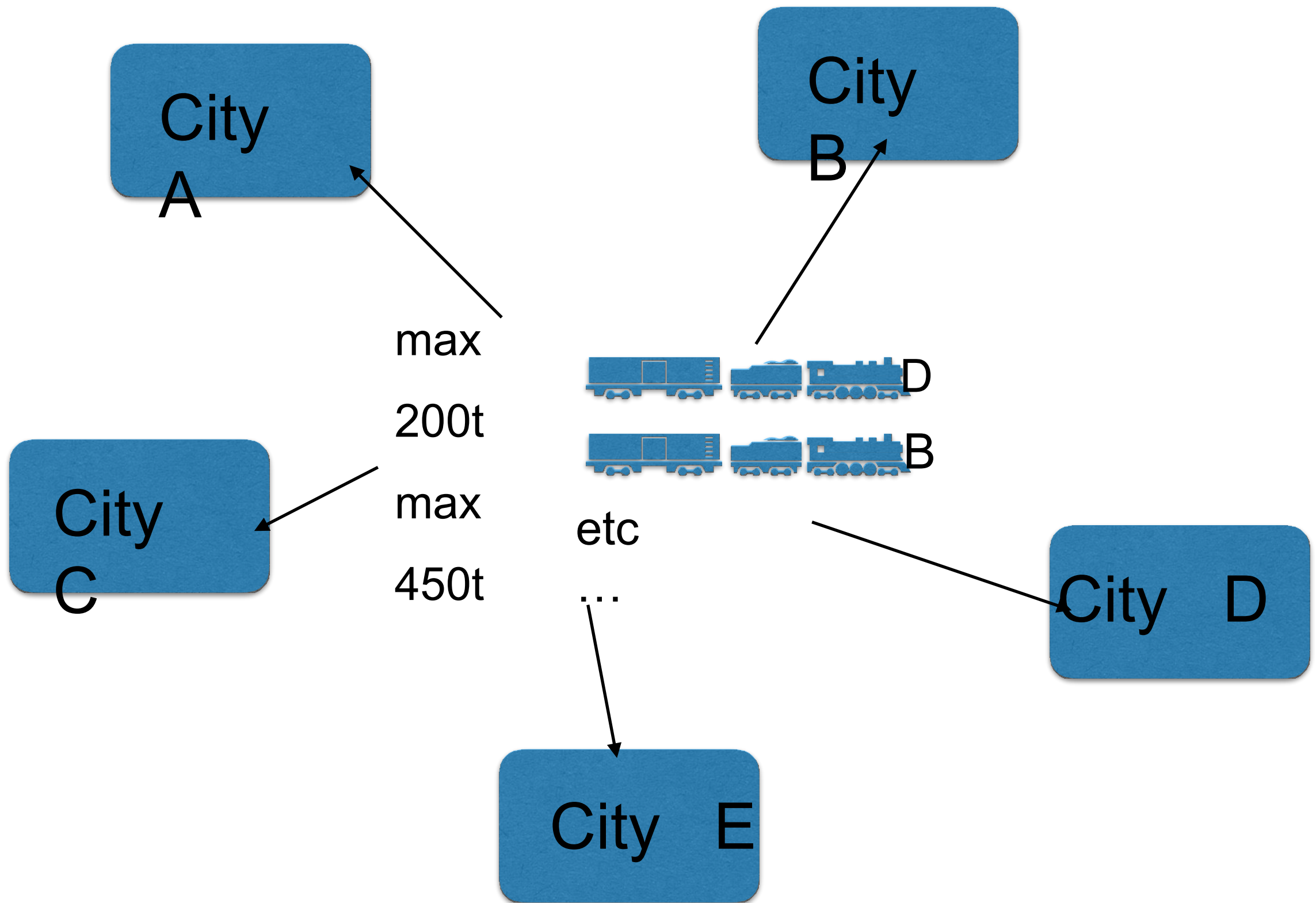
- an (encapsulated) internal state
- well-defined external interactions and state changes
- a separate identity from other objects

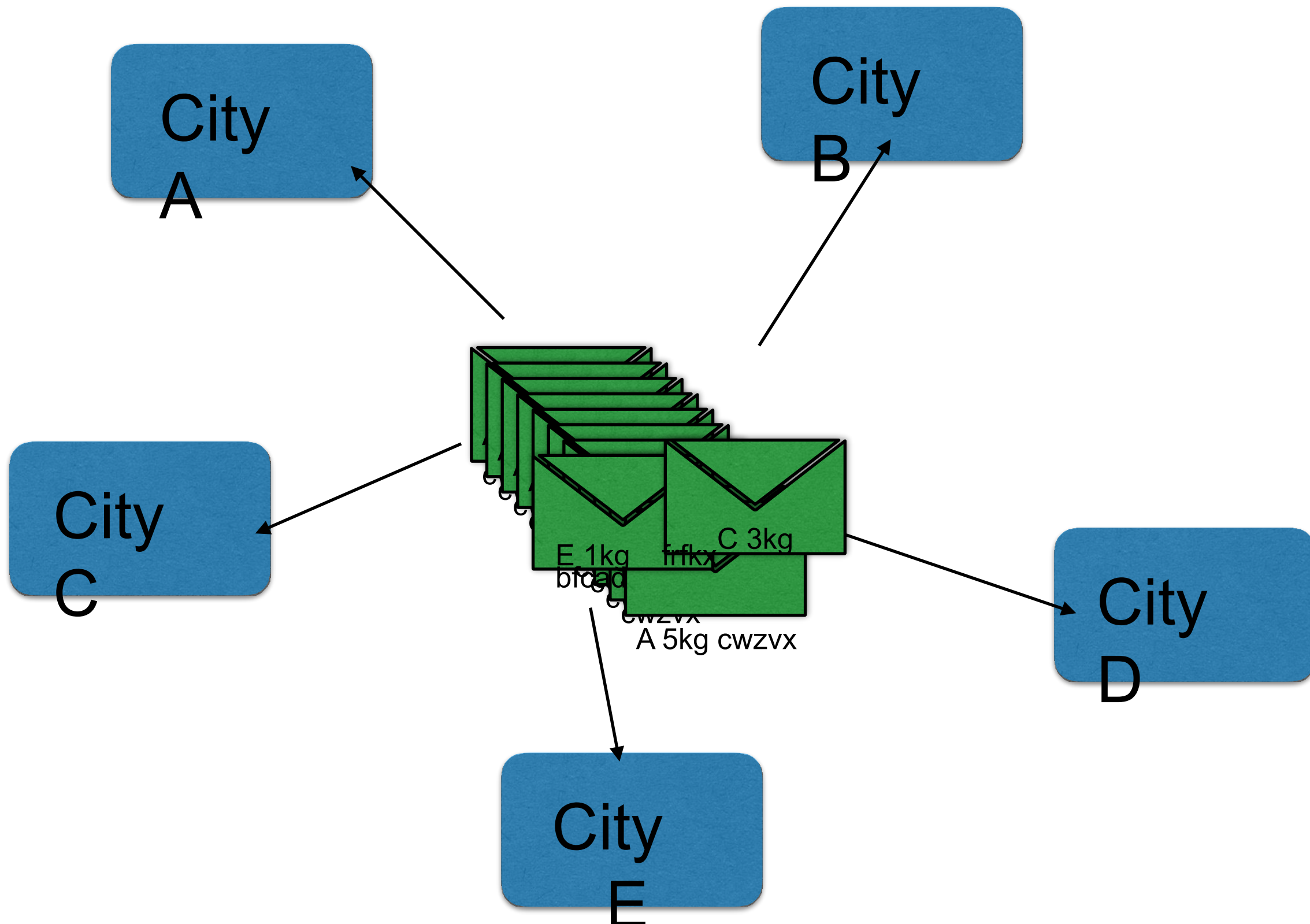
A class describes the common structure and behaviour of the objects belonging to

Exercise

Exercise: a freight station







Design an OO

model for the

classes, objects, interfaces, public/private, which methods/state
station

but no implementation!



a random train arrives, is loaded with correct mail, leaves, and repeat

max
200t

